

Rapport — Jour 2

Étage hors-contexte : grammaires & automate à pile

Binôme : DOSSOU Persévérance & AGBOGBA Silas • Modules : `formlang/pda.py`, `cfg.py`

A. Réponses théoriques

Q2.1 (1,5) — non-régularité de $D = \{[n]^n \mid n \geq 0\}$ par pompage

Par l'absurde. Supposons D régulier. Par le lemme de pompage, il existe $p \geq 0$ tel que tout mot $w \in D$ avec $|w| \geq p$ s'écrit $w = xyz$ avec $|xy| \leq p$, $y \neq \varepsilon$, et $xy^iz \in D$ pour tout $i \geq 0$.

Choix du mot. On prend $w = [^p]^p$ (longueur $2p \geq p$, donc $w \in D$).

Discussion de toute découpe admissible. Comme $|xy| \leq p$, le préfixe xy est entièrement contenu dans le bloc des p premières occurrences de $[$ (le mot ne contient aucun $]$ avant la position p). Donc y est nécessairement composé uniquement de $[$: $y = [^k$ avec $k \geq 1$ (puisque $y \neq \varepsilon$). Ceci est vrai **quelle que soit** la découpe x, y, z satisfaisant $|xy| \leq p$: il n'y a aucune liberté sur la nature de y , seulement sur sa position et sa longueur $k \in \{1, \dots, p\}$.

Mot pompé hors de D . Pompons avec $i = 2$:

$$xy^2z = [^{p+k}]^p.$$

Ce mot possède $p + k$ crochets ouvrants et seulement p fermants. Comme $k \geq 1$, on a $p + k \neq p$, donc $xy^2z \notin D$.

Conclusion. Ceci contredit la conclusion du lemme de pompage ($xy^iz \in D$ pour tout i). Donc l'hypothèse de départ est fausse : D n'est pas régulier.

Conséquence pour le projet. Un AFD (jour 1) ne dispose que d'une mémoire **finie** (un nombre fini d'états) : il ne peut pas compter une profondeur d'imbrication non bornée. Le `SingularityDetector` de jour 1 est donc structurellement incapable de vérifier que les délimiteurs `[ ... ] / ( ... )` sont bien imbriqués, quel que soit le nombre d'états qu'on lui donnerait : c'est précisément ce que D (le langage de Dyck simplifié) met en évidence. Il faut une mémoire non bornée mais disciplinée — la **pile** du `DelimiterPDA`.

Q2.2 (0,5) — grammaire des prompts bien parenthésés

$$S \rightarrow SS \mid [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$$

(implémentée dans `balanced_cfg()`, `formlang/cfg.py`).

Exemple de dérivation pour le mot `a[o]` :

$$S \Rightarrow (S) \Rightarrow (SS) \Rightarrow (aS) \Rightarrow (a[S]) \Rightarrow (a[o])$$

Q2.3 (0,5) — ambiguïté

G est **ambiguë**. Le mot `aaa` (obtenu via deux applications de $S \rightarrow SS$) admet au moins deux arbres de dérivation distincts.

Arbre 1 — association à gauche $(aa)a$.

$$S \rightarrow SS \rightarrow (SS)S \rightarrow (aa)a$$

racine S avec fils gauche $S \rightarrow SS \rightarrow aa$, fils droit a .

Arbre 2 — association à droite $a(aa)$.

$$S \rightarrow SS \rightarrow S(SS) \rightarrow a(aa)$$

racine S avec fils gauche a , fils droit $S \rightarrow SS \rightarrow aa$.

Les deux arbres dérivent le même mot `aaa` avec une structure différente (le regroupement des deux S dans $S \rightarrow SS$ n'est pas déterminé) $\Rightarrow G$ est **ambiguë**.

Grammaire non ambiguë équivalente (association à droite imposée) :

$$S \rightarrow TS \mid T \qquad T \rightarrow [S] \mid (S) \mid a \mid o \mid r \mid \varepsilon$$

Chaque T est un bloc atomique ; S les concatène en séquence associée à droite de façon unique, ce qui supprime le choix de regroupement responsable de l'ambiguïté.

B. Exercices de programmation

E2.1

`DelimiterPDA.accepts` (`formlang/pda.py`) : simulation par une pile Python (`list`). Un jeton de `ignore` (`a, o, r`) est sauté ; un ouvrant est empilé ; un fermant vérifie que le sommet de pile est bien l'ouvrant correspondant (`self.match`), sinon rejet immédiat ; acceptation ssi la pile est vide en fin de lecture. *Test* `test_delimiters_pda` : *passé*.

E2.2

`CFG.generate(max_len)` (`formlang/cfg.py`) : parcours en largeur des formes sententielles à partir de (S) , développement du non-terminal le plus à gauche (dérivation gauche), collecte des mots purement terminaux de longueur $\leq \text{max_len}$. *Piège de l'énoncé traité* : avec $S \rightarrow SS$, les formes purement non-terminales $(S, SS, SSS, \dots)$ ont 0 terminal et ne seraient jamais élaguées par la seule borne sur les terminaux $\Rightarrow$ on borne **aussi** le nombre total de symboles de la forme sententielle (`max_symbols = max_len + #non-terminaux + 2`), ce qui n'élague aucun mot de longueur $\leq \text{max_len}$. Un ensemble `visited` évite de retraiter deux fois la même forme. *Test `test_cfg_engendre_des_mots_equilibres` : passé.*

Sortie de référence obtenue

```
$ pytest -q tests/test_regular.py tests/test_cfg_nerode.py
8 passed, 1 failed
```

(L'unique échec, `test_nerode_trois_classes`, relève du Jour 5 — hors périmètre de cette fiche.)